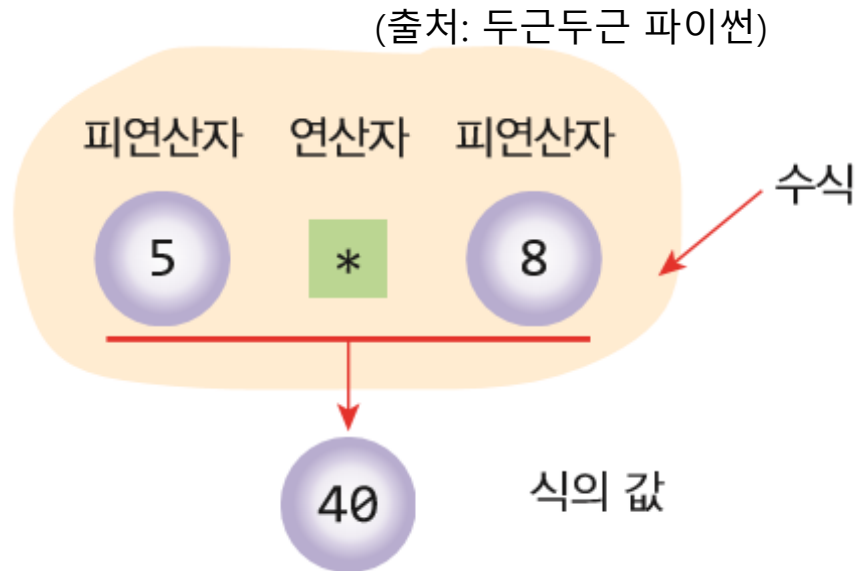


CHAP 3: 연산자

산술 연산자,
복합 연산자,
관계 연산자,
논리 연산자,
연산자 우선순위

산술 연산자

- 수식(expression) = 피연산자들과 연산자의 조합
- 연산자(operator): 연산을 나타내는 기호
- 피연산자(operand): 연산의 대상이 되는 값



산술 연산자

- 덧셈, 뺄셈, 곱셈, 나눗셈, 나머지 연산
- 피연산자 모두 정수형이 아닐 경우, 결과는 항상 실수이다.

operator	meaning	예
+	덧셈 또는 + 부호	+10 (양수의 경우 +부호 생략 가능) $15 + 20$
-	뺄셈 또는 - 부호	-10 $2.5 - 1.5$
*	곱셈	$10 * 20 = 200$
/	나눗셈(실수)	$1/2 = 0.5$ (결과가 항상 실수)
%	나눈 후 나머지 (modulo)	$9 \% 4 = 1$ (9를 4로 나눈 나머지)
//	나눈 후 몫 (floor division)	$15//4 = 3$ (15를 4로 나눈 몫)

산술 연산자

- 한 줄에 1개의 명령어를 작성한다.
- 여러 줄에 걸쳐 명령어를 작성할 때는 **backslash(\)** 를 사용.
(~~W~~ 또는 \로 표시됨).

● 예)

```
>>> a = 1 + 2 + 3 + W Enter
      4 + 5 + 6 + W Enter
      7 + 8 + 9 Enter
>>> a
45
```

- 자료형에 사용되는 (...), [...], {...} 등의 괄호 내부에서는 **backslash**없이 여러 줄에 걸쳐 작성 가능. 함수의 ()안에서도 가능.

● 예)

```
>>> print( 1 + 2 + 3 + Enter
          4 + 5 + 6 + Enter
          7 + 8 + 9 ) Enter
45
>>>
```

산술 연산자

- 한 줄에 1개의 명령어를 작성한다.
- 한 줄에 2개 이상의 명령어를 작성할 때는 **semicolon(;)**을 사용하여 구분한다.
 - 예) 한 줄에 여러 개의 명령어 작성하기.

```
>>> a = 1; b = 2; c = 3   
>>> print(a, b, c)   
1 2 3  
>>>
```

한 줄에 명령어 3개 작성.

산술 연산자

- 나눗셈의 몫 연산
 - // : 나눗셈의 몫
 - / : 나눗셈 (결과는 항상 실수)

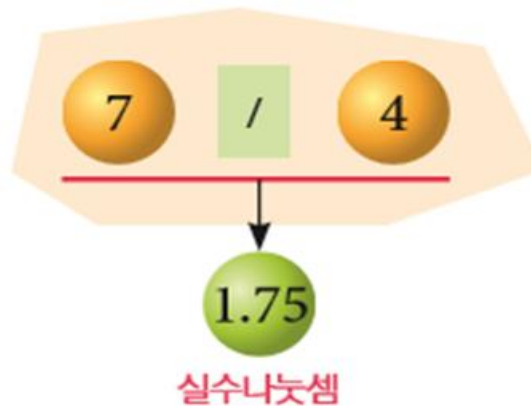
```
>>> 7 / 4 # 피연산자가 정수인 경우
```

```
1.75
```

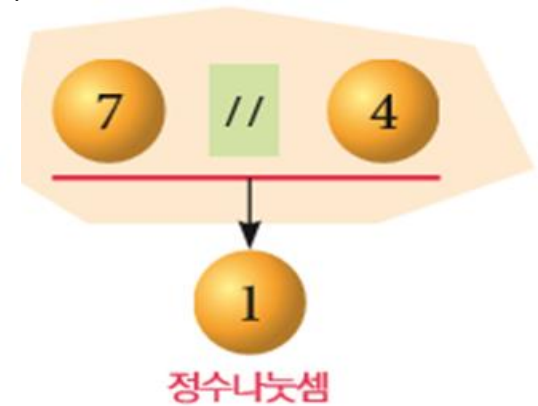
```
>>> 7 // 4
```

```
1
```

```
>>>
```



(출처: 어서와 파이썬은 처음이지)



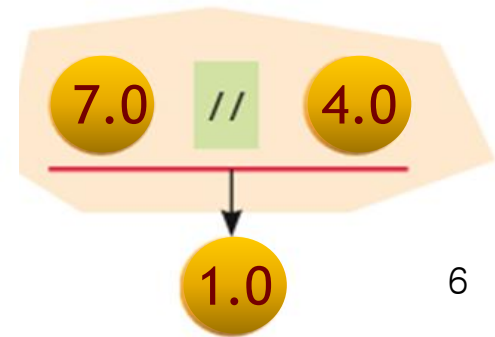
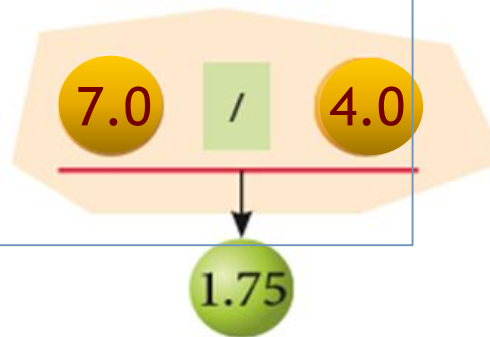
```
>>> 7.0 / 4.0 # 피연산자가 실수인 경우
```

```
1.75
```

```
>>> 7.0 // 4.0
```

```
1.0
```

```
>>>
```

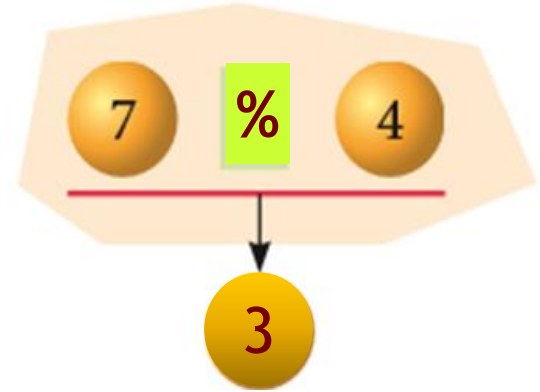


산술 연산자

- 나머지 연산자 (모듈로 Modulo 연산자)

- % 연산자
- 나눗셈이 수행된 후 나머지 값을
결과로 산출

```
>>> e = 7 % 4  
>>> e  
3  
>>>
```



- 예) 숫자가 짝수(홀수)인지 나머지 연산자를 사용하여 확인한다.

- 짝수 % 2는 항상 0.
- 홀수 % 2는 항상 1.

```
>>> n1 = 30 ; n2 = 35  
>>> n1 % 2    # 짝수  
0  
>>> n2 % 2    # 홀수  
1  
>>>
```

산술 연산자

- 수식의 계산 결과 출력

- `print()` 함수 인수에 수식을 전달하면 계산결과를 출력.

```
>>> print(6+4, 6-4, 6*4, 6/4) # 수식의 계산결과를 출력
10 2 24 1.5
>>>
```

`>>> print(` `)`

↓ 인수 $6 + 4, 6 - 4, 6 * 4, 6 / 4$ 수식

10 2 24 1.5

출력 결과

```
>>> print("6+4=", 6+4) # 계산식과 계산결과 출력
6+4= 10
>>>
```


산술 연산자

- 우선순위
 - 산술 연산자의 우선순위는 괄호가 가장 높다.
곱셈(또는 나눗셈)이 그 다음,
덧셈(또는 뺄셈)이 가장 마지막으로 수행.
 - 덧셈(또는 뺄셈)끼리 나오거나 곱셈(또는 나눗셈)끼리 나오면
왼쪽에서 오른쪽으로 계산이 진행됨

● 예)

$$\underbrace{x + y}_{\textcircled{2}} * z \quad \textcircled{1}$$

$$\underbrace{(x + y)}_{\textcircled{1}} * \underbrace{z}_{\textcircled{2}}$$

산술 연산자

- 예) 평균 구하기.
 - 숫자를 더하는 연산을 나눗셈 연산보다 먼저 하기 위해 괄호로 묶어 계산하고 3으로 나눈다.

3개 시험 성적의 평균 구하기

- 수학: 26점
- 영어: 54점
- 역사: 96점

```
>>> print( (26+54+96)/3 )   
58.666666666666664  
>>> print( (26+54+96)//3 )   
58  
>>> print( int( (26+54+96)/3 ) )   
58  
>>>
```

```
>>> a = 26; b = 54; c = 96  
>>> 평균 = (a+b+c)/3  
>>> 평균  
58.666666666666664  
>>> 평균 = int(평균)  
>>> print(평균)  
58  
>>>
```

평균이 정수 값으로 필요할 때,
// 연산을 사용하거나 또는 /연산 후 int()함수로 형 변환한다.

산술 연산자

- round() 실수를 반올림하는 내장 함수.
 - round(실수)
 - 자릿수를 지정하지 않으면, 반올림하여 정수 반환.
 - round(실수, 자릿수)
 - 자릿수: 반올림 후의 소수점 이하 자릿수 지정
 - 반올림하여 지정된 소수점 이하 자릿수까지의 실수 반환.

```
>>> num= (26+54+96)/3
>>> num
58.666666666666664
>>> y= round(num) #반올림하여 정수 반환
>>> y
59
>>> print( y )
59
>>> print( round(num, 2) ) # 반올림 후 소수점 이하 자릿수 2
58.67
>>>
```

산술 연산자

- 예) 1000초를 분 값과 나머지 초 값으로 **변환**하기

test.py

```
seconds = 1000
```

```
# 초 단위 값을 분 값으로 계산(변환)하기
```

```
mins = seconds // 60
```

```
# 분 값으로 변환 후 나머지 초 값?
```

```
remainder = seconds % 60
```

```
# 변수와 문자열을 출력
```

```
print( mins, '분', remainder, '초')
```

출력 화면 (python shell 창)

16 분 40 초

산술 연산자

- 예) 화씨 온도 100도를 섭씨 온도로 바꾸기.



$$C = (F - 32) * \frac{5}{9}$$

test.py

```
ftemp = 100 # 화씨 온도
ctemp = (ftemp-32) * 5/9 # 섭씨 온도 ctemp= 37.77777777777778
print("화씨 온도:", ftemp)
print("섭씨 온도:", ctemp)

# 반올림하여 출력.
ctemp = round(ctemp, 1) → 37.8
print("섭씨 온도:", ctemp)
```

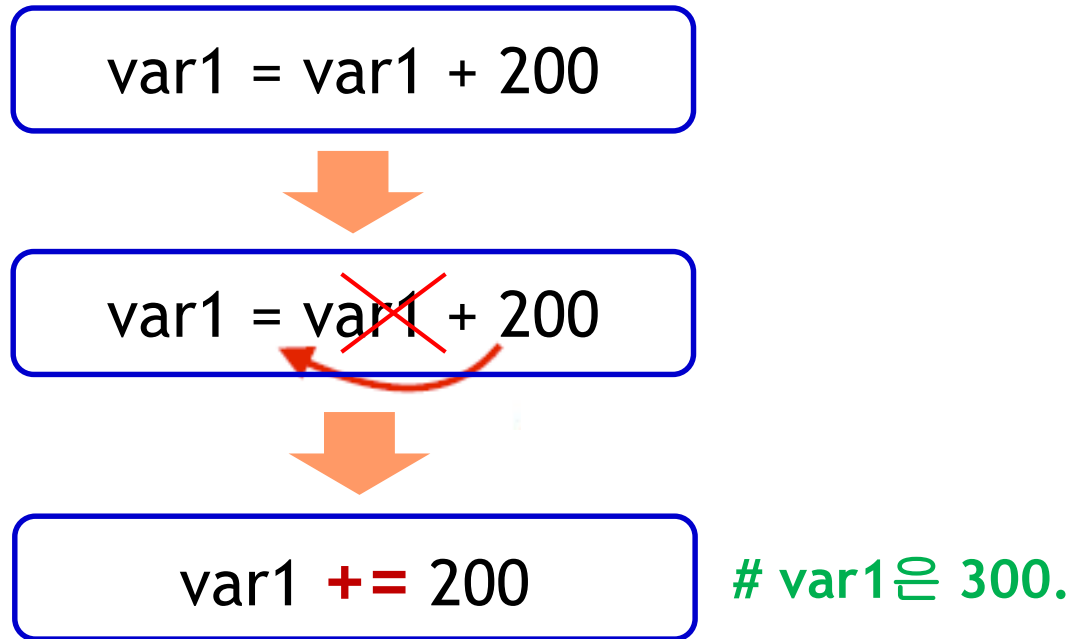
반올림 처리
(소수점 이하 첫째 자리까지)

실행 화면 (python shell 창)

```
화씨온도: 100
섭씨온도: 37.77777777777778
섭씨온도: 37.8
```

복합 연산자

- 복합 연산자(compound operator)
 - 연산과 할당을 동시에 표현.
변수를 연산에 사용하고 결과를 그 변수에 다시 할당.
 - **+=** 처럼 할당 연산자와 다른 연산자를 합쳐 놓은 연산자
- 예) `var1=100` 이라 할때, 변수 `var1`을 200만큼 증가시키기.



복합 연산자

- 예) 변수 **a**에 1~10 까지 합계 할당하기.
a = 1+2+3++9+10 # 이 계산식을 바꾸어 보자.
 - 변수 **a**에 0를 할당.
 - 여기에 숫자 1을 +연산하고 결과를 변수 **a**에 다시 할당한다.
차례로 숫자 10까지 이렇게 반복한다.

```
a=0
a=a+1
a=a+2
a=a+3
a=a+4
a=a+5
a=a+6
a=a+7
a=a+8
a=a+9
a=a+10
```

```
>>> a=0
>>> a +=1
>>> a +=2
>>> a +=3
>>> a +=4
>>> a +=5
>>> a +=6
>>> a +=7
>>> a +=8
>>> a +=9
>>> a +=10
>>> a
55
```

```
>>> a=1+2+3+4+5+6+7+8+9+10
>>> print(a)
55
>>>
```

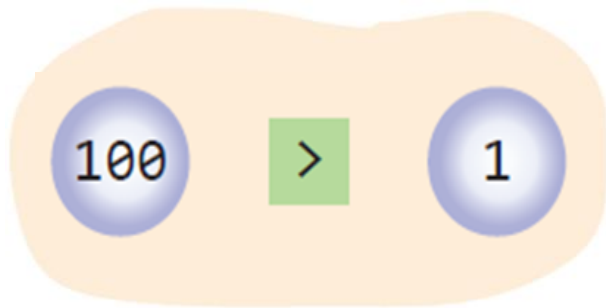
관계 연산자(Relational Operators)

- 관계 연산자
 - 두 값을 비교하는 연산자
 - 결과는 불리언 값. **True**(참) 또는 **False**(거짓).
 - 주로 조건문(**if**)이나 반복문(**for**, **while**)에서 사용
 - 조건이 참인지 거짓인지 판단할 때 사용.

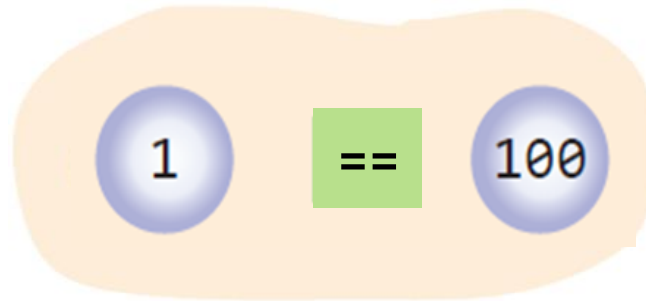
operator	description	example
==	equal	5 == 7 # False
!=	not equal	5 != 7 # True
>	greater than	5 > 7 # False
<	less than	5 < 7 # True
>=	greater than or equal	5 >= 7 # False
<=	less than or equal	5 <= 7 # True

관계 연산자

- 관계 연산자의 결과값
 - 관계 수식은 참(True)이나 거짓(False)을 생성



계산결과값: True



계산결과값: False

관계 연산자

- 예) 어떤 숫자의 홀수/짝수 확인을 위해 관계연산자 사용한다.
 - 짝수 % 2는 항상 0
 - 홀수 % 2는 항상 1.

```
>>> 10% 2 == 0  # 나머지 0. True. 짝수
True
>>> 10% 2 == 1  # 나머지 0. False. 홀수 아님
False
>>> 13% 2 == 0  # 나머지 1. False. 짝수 아님
False
>>> 13% 2 == 1  # 나머지 1. True. 홀수
True
>>> evenNum= 6
>>> evenNum%2 == 0
True
>>> evenNum%2 == 1
False
>>>
```

논리 연산자

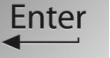
- 논리 연산자의 종류
 - 복잡한 조건을 표현하려면 논리연산자를 사용.
 - 여러 조건식을 조합해서 복합 표현식을 구성할 수 있다.
 - 몇 개의 조건식을 조합하여 명령문의 수행여부를 결정할 때 사용

operator	description	example
and	logical and. ~이고 그리고	모두 True이어야 True
or	logical or. ~이거나 또는	하나라도 True이면 True
not	negates the truth value 부정	참이면 거짓. 거짓이면 참.

논리 연산자

- 논리 연산자의 연산은 다음 진리표(truth table)에 의한다.
 - **and** 연산

입력값		x 그리고 y
x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True

```
>>> False and False   
False  
>>> False and True   
False  
>>> True and False   
False  
>>> True and True   
True  
>>>
```

논리 연산자

- 논리 연산자의 연산은 다음 진리표(truth table)에 의한다.

- **or** 연산

입력값		x 또는 y
x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True

```
>>> False or False
```



```
False
```

```
>>> False or True
```



```
True
```

```
>>> True or False
```



```
True
```

```
>>> True or True
```



```
True
```

```
>>>
```

논리 연산자

- 논리 연산자의 연산은 다음 진리표(truth table)에 의한다.
- **not** 연산

입력값	X가 아니다
x	not x
False	True
True	False

```
>>> not False   
True  
>>> not True   
False  
>>>
```

논리 연산자

- 예)

```
>>> a = 320
>>> True and a==300 # True and False
False
>>> (a > 100) and (a%2 == 0) # True and True
True
>>> (a == 100) or (a == 200) # False or False
False
>>> not (a == 100) # not False
True
>>>
```

논리 연산자

- 예) 복합 조건

조건 1

조건 2

나이가 10살 이상이고, 그리고 키가 165cm 이상이면
→ 놀이기구를 탈 수 있다.

(나이가 10살 이상이다) **and** (키가 165cm 이상이다)
→ 놀이기구를 탈 수 있다.



(출처: 두근두근 파이썬)

```
>>> age = 15
```

```
>>> height=160
```

```
>>> age >= 10 and height >= 165
```

Enter

나이가 10살 이상

그리고

키가 165cm 이상

False

```
>>>
```


논리 연산자

- 예) 변수 `year`가 400으로 나눠 떨어지는지 확인

```
>>> year= 2022
>>> ( year%400 ) == 0
False
>>> year%400 == 0
False
```

- 예) 4로는 나눠 떨어지고, 100으로 나눠 떨어지지 않는지 확인

```
>>> year= 2020
>>> ( year%4 == 0 ) and ( year%100 !=0)
True
>>>
```

연산자 우선순위

우선순위

()	anything in brackets is done first	Highest
**	exponentiation	
-x, +x	arithmetic operators	
*, /, %, //	arithmetic operators	
+, -	arithmetic operators	
<, >, <=, >=, !=, ==	relational operators	
=, +=, -=, *=, etc	assignment operators	
not	logical operator	
and	logical operator	
or	logical operator	Lowest

- 같은 우선순위를 갖는 operator는 왼쪽부터 계산한다.
- 애매하면 괄호 () 를 치자 (예: $(2**2)**3 = 4**3 = 64$).